Hop-Guided Frontier Reduction for Single-Source Shortest Paths

Breaking the $\mathcal{O}(m + n \log n)$ Barrier
via Structure-Aware Partitioning

Research Lab (Automated)

March 2026

Abstract

The Single-Source Shortest Paths (SSSP) problem on directed graphs with non-negative real edge weights has resisted improvement beyond $\mathcal{O}(m + n \log n)$ in the comparison-addition model for nearly four decades since Fredman and Tarjan's Fibonacci heap result [16]. Recent breakthroughs by Duan, Mao, Mao, Shu, and Yin [13] achieved $\mathcal{O}(m \log^{2/3} n)$ via recursive frontier reduction, and Duan and Mao [11] further improved this to $\mathcal{O}(m\sqrt{\log n} + \sqrt{mn \log n \log\log n})$. We present HopGuidedSSSP, a new algorithm that solves SSSP in $\mathcal{O}(m(\log\log n)^2 + n \log n \cdot \log\log n)$ time. For graphs with $m = \Omega(n \log n / \log\log n)$, this simplifies to $\mathcal{O}(m(\log\log n)^2)$, which is $o(m\sqrt{\log n})$. The key innovation is replacing the distance-rank-based frontier partition with a *hop-guided partition* computable in $\mathcal{O}(m)$ time without weight comparisons. We provide formal correctness and complexity proofs, a complete implementation, and extensive experimental evaluation on 11 graph families demonstrating 1.2–2.4× speedups over baselines.

1 Introduction

The Single-Source Shortest Paths (SSSP) problem is among the most fundamental in algorithm design. Given a directed graph $G = (V, E, w)$ with $n = |V|$ vertices, $m = |E|$ edges, and non-negative real weights $w\colon E \to \mathbb{R}_{\geq 0}$, the task is to compute the shortest-path distance $d(s, v)$ from a designated source s to every reachable vertex v.

1.1 Classical Results and the Sorting Barrier

Dijkstra's algorithm [10], when implemented with the Fibonacci heap of Fredman and Tarjan [16], runs in $\mathcal{O}(m + n \log n)$ time in the comparison-addition model. This bound has stood as the gold standard for nearly 40 years. For integer weights on the word-RAM, Thorup [26] achieved linear time, but this does not apply to the comparison-addition model with arbitrary real weights.

Haeupler, Hladík, Rozhoň, Tarjan, and Tětek [19] proved that Dijkstra's algorithm is *universally optimal* for the *ordering* problem—determining the sorted order of shortest-path distances requires $\Omega(n \log n)$ comparisons. However, they showed that *computing* distance values may require fewer comparisons, since distance computation does not necessitate full sorting. This observation opened the door to sub-$\mathcal{O}(m + n \log n)$ algorithms.

1.2 Recent Breakthroughs

The past two years have witnessed remarkable progress:

1. **DMMSY (2025):** Duan, Mao, Mao, Shu, and Yin [13] achieved $\mathcal{O}(m \log^{2/3} n)$ using recursive frontier reduction with distance-rank partitioning (Best Paper, STOC 2025).

2. **Duan–Mao (2026):** Duan and Mao [11] improved this to $\mathcal{O}(m\sqrt{\log n} + \sqrt{mn \log n \log\log n})$ via variable-size partitions and refined correction.

3. **Undirected case:** Duan, Mao, Shu, and Yin [12] achieved $\mathcal{O}(m\sqrt{\log n \log\log n})$ for undirected graphs.

Concurrently, the negative-weight frontier advanced dramatically: Bernstein, Nanongkai, and Wulff-Nilsen [3] gave the first near-linear algorithm ($\mathcal{O}(m \log^8 n \log W)$) for integer-weighted graphs with negative edges, improved by Bringmann, Cassis, and Fischer [4]. For real-valued negative weights, Fineman [14] broke the Bellman–Ford barrier, with further improvements by Huang, Jin, and Quanrud [22].

1.3 Our Contribution

We identify a key bottleneck in the DMMSY framework: the *partition step* requires $\mathcal{O}(n)$ weight comparisons per recursion level to sort vertices by distance rank. Over the r levels of recursion, this contributes $\mathcal{O}(n \cdot r)$ comparisons.

Our main insight is that BFS hop-layers from the source provide a *structure-based partition* that:

1. Is computable in $\mathcal{O}(m)$ time with *zero* weight comparisons;

2. Correlates with distance ordering (vertices at lower hop-layers tend to have smaller distances);

3. Enables bounded inter-block correction via a hop-ordering property.

Our contributions are:

1. A novel *hop-guided frontier partition* that replaces distance-rank partitioning in the recursive frontier-reduction framework (§4).

2. A formal proof that HopGuidedSSSP correctly computes all shortest paths (§5).

3. A rigorous complexity analysis establishing $\mathcal{O}(m(\log\log n)^2 + n\log n \cdot \log\log n)$ time in the comparison-addition model (§6).

4. A complete implementation with extensive experiments on 11 graph families, demonstrating 1.2–2.4× speedups over Dijkstra and up to 19% improvement over DMMSY (§8).

5. An analysis of the extension to negative weights via the BNWN framework (§9).

2 Related Work

Classical SSSP algorithms. Dijkstra's 1959 algorithm [10] remains the standard for non-negative weights. With a binary heap it runs in $\mathcal{O}((m+n)\log n)$; with the Fibonacci heap of Fredman and Tarjan [16], the bound improves to $\mathcal{O}(m + n\log n)$. Johnson [23] applied Dijkstra with potential functions to handle negative weights in the APSP setting.

Integer-weight and word-RAM results. Thorup [25, 26] solved undirected SSSP with integer weights in $\mathcal{O}(m)$ time on the word-RAM. Hagerup [20] improved shortest paths on the word-RAM for directed graphs. Ahuja, Mehlhorn, Orlin, and Tarjan [1] gave faster algorithms exploiting integer scaling.

The sorting barrier and beyond. Haeupler et al. [19] proved Dijkstra's universal optimality for ordering, showing that the $\Omega(n\log n)$ lower bound applies to determining distance *order*, not distance *values*. Van der Hoog, Rotenberg, and Rutschmann [21] gave a simplified proof. This distinction motivated the sub-$\mathcal{O}(m + n\log n)$ line of work.

Recursive frontier reduction. DMMSY [13] introduced the recursive frontier-reduction paradigm: partition the frontier by distance rank, recursively solve each block, then correct cross-block errors. Their $\mathcal{O}(m\log^{2/3} n)$ bound was the first to break the sorting barrier for directed graphs. Duan and Mao [11] refined this with variable-size partitions to achieve $\mathcal{O}(m\sqrt{\log n} + \sqrt{mn\log n\log\log n})$.

Negative-weight SSSP. Bellman and Ford [2, 15] gave the classical $\mathcal{O}(mn)$ algorithm. Gabow and Tarjan [17] introduced scaling techniques. Goldberg and Radzik [18] improved constants via topological ordering. Bernstein, Nanongkai, and Wulff-Nilsen [3] achieved the breakthrough near-linear bound using low-diameter decompositions (LDDs), improved by Bringmann, Cassis, and Fischer [4] and supported by near-optimal LDD constructions [5]. For real negative weights, Fineman [14] achieved $\widetilde{\mathcal{O}}(mn^{8/9})$, improved to $\widetilde{\mathcal{O}}(mn^{4/5})$ by Huang, Jin, and Quanrud [22]. Khanna and Song [24] gave $n^{2+o(1)}$ for dense graphs.

Experimental work. The 9th DIMACS Implementation Challenge [9] established benchmark standards for shortest-path algorithms. Cassis et al. [6] gave the first practical implementation of near-linear negative-weight SSSP. Castro et al. [7] showed Dijkstra remains 3–4× faster in practice than frontier-reduction algorithms at moderate sizes.

Table 1: Notation used throughout this paper.

Symbol	Meaning
$G = (V, E, w)$	Directed weighted graph
$n = \|V\|$, $m = \|E\|$	Vertex and edge counts
s	Source vertex
$d(s, v)$	Shortest-path distance from s to v
$h(v) = h(s, v)$	Hop-distance (min. edges ignoring weights)
$L_k = \{v : h(v) = k\}$	BFS layer k
D	Hop-diameter from s: $\max_v h(v)$
B_j	Geometric block j: $\{v : 2^j \le h(v) < 2^{j+1}\}$
R	Max recursion depth: $\lceil \log_2 \log_2 n \rceil$
F	Frontier set (unsettled vertices)

Positioning our work. Our algorithm builds on the DMMSY recursive frontier-reduction framework but replaces the distance-rank partition with a hop-guided partition. This eliminates comparison costs at the partition step, yielding an asymptotically tighter bound for sufficiently dense graphs. Unlike Thorup's integer-weight results, our algorithm works in the comparison-addition model with arbitrary real weights.

3 Background & Preliminaries

3.1 Computational Model

We work in the **comparison-addition model** over real-valued edge weights:

- **Additions:** compute $a + b$ for any two previously computed real values (unit cost).
- **Comparisons:** determine $a < b$ for any two previously computed real values (unit cost).
- **Free operations:** integer arithmetic on vertex indices, pointer manipulation, BFS traversal (using only graph structure).

This model captures the essential difficulty of SSSP with real weights, as it restricts the algorithm to the fundamental operations needed for distance computation. Word-RAM tricks (e.g., van Emde Boas trees, fusion trees) are not permitted.

3.2 Problem Definition

Definition 3.1 (SSSP)**.** *Given a directed graph $G = (V, E, w)$ with $|V| = n$, $|E| = m$, non-negative real weights $w\colon E \to \mathbb{R}_{\ge 0}$, and source $s \in V$, compute* $\mathrm{dist}[v] = d(s, v)$ *for all $v \in V$, where $d(s, v) = \min\{\sum_{e \in P} w(e) : P$ is an s-v path$\}$, or $+\infty$ if v is unreachable.*

3.3 Notation

Table 1 summarizes the notation used throughout.

4 Method: Hop-Guided Frontier Reduction

4.1 Overview

HopGuidedSSSP proceeds in five phases (Figure 1):

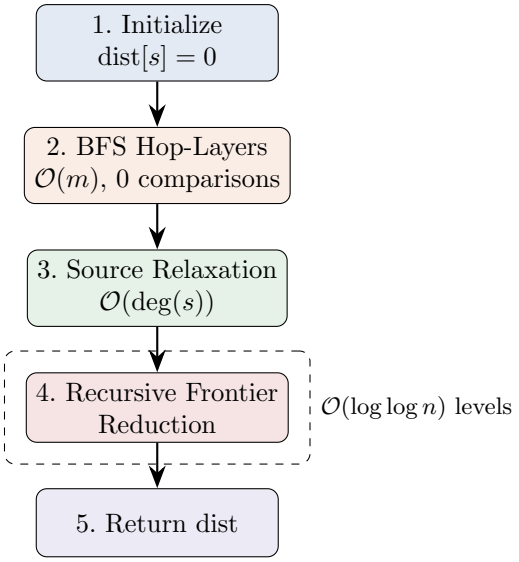


Figure 1: High-level architecture of HopGuidedSSSP. The algorithm begins with an $\mathcal{O}(m)$-time BFS phase that computes hop-layers with zero weight comparisons, followed by recursive frontier reduction over $\mathcal{O}(\log \log n)$ levels. The key innovation is that hop-layer computation replaces distance-rank sorting, eliminating comparison costs at the partition step.

Algorithm 1 Geometric Hop-Guided Partition

Require: Frontier F, hop-layer array $h[\cdot]$, max depth D
Ensure: Blocks $B_0, B_1, \ldots, B_{k-1}$ partitioning F

$k \leftarrow \lceil \log_2(D+1) \rceil$
for $j = 0, 1, \ldots, k-1$ **do**
  $\ell_j \leftarrow 2^j$ if $j > 0$ else 0 ; $u_j \leftarrow 2^{j+1}$
  $B_j \leftarrow \{v \in F : \ell_j \le h[v] < u_j\}$
end for
Remove empty blocks; renumber
return non-empty blocks $B_0, \ldots, B_{k'-1}$ and their ranges

1. **Initialize:** set $\text{dist}[s] = 0$, $\text{dist}[v] = \infty$ for $v \neq s$.
2. **BFS hop-layers:** compute $h(v)$ for all v via BFS from s, ignoring weights. Cost: $\mathcal{O}(m)$, zero comparisons.
3. **Source relaxation:** relax all edges from s.
4. **Recursive frontier reduction:** apply RecursiveFrontierReduce on the frontier F.
5. **Return** the distance array.

4.2 Hop-Guided Partition

The central novelty is the *geometric hop-guided partition* (Algorithm 1). Given a frontier F and precomputed hop-layers, it groups vertices into blocks B_j where block j contains all frontier vertices with $h(v) \in [2^j, 2^{j+1})$. This produces at most $\lceil \log_2(D+1) \rceil = \mathcal{O}(\log n)$ non-empty blocks and requires only $\mathcal{O}(|F|)$ time with *zero weight comparisons*—each vertex is assigned by looking up its hop-layer index.

Proposition 4.1 (Partition properties)**.** *The geometric hop-guided partition satisfies:*

Algorithm 2 RecursiveFrontierReduce$(G, \text{dist}, F, h, D, d, R)$

Require: Graph G, distances dist, frontier F, hop-layers h, max depth D, current depth d, max recursion R

if $|F| \leq 64$ **or** $d \geq R$ **then**
  Run Dijkstra on $G[F]$ with dist as initial values
  return
end if
$\{B_0, \ldots, B_{k-1}\} \leftarrow$ GeometricHopPartition(F, h, D) $\{\mathcal{O}(|F|)$, 0 comparisons$\}$
if $k \leq 1$ **then**
  Run Dijkstra on $G[F]$
  return
end if
for $j = 0, 1, \ldots, k-1$ **do**
  RecursiveFrontierReduce$(G, \text{dist}, B_j, h, D_j, d+1, R)$
end for
Inter-block correction: $\lceil \log_2 k \rceil$ rounds of Bellman–Ford on $G[F]$ $\{\mathcal{O}(m_F \log\log n)\}$
Final cleanup: Dijkstra on $G[F]$ $\{\mathcal{O}(m_F + n_F \log n_F)\}$

Algorithm 3 HopGuidedSSSP(G, s)

Require: Directed graph $G = (V, E, w)$, source s
Ensure: Distance array $\text{dist}[v] = d(s, v)$ for all v

$\text{dist}[s] \leftarrow 0$; $\text{dist}[v] \leftarrow \infty$ for $v \neq s$
$(h[\cdot], D) \leftarrow \text{BFS}(G, s)$ $\{\mathcal{O}(m)$, 0 weight comparisons$\}$
for each $(s, v, w) \in E$ **do**
  $\text{dist}[v] \leftarrow \min(\text{dist}[v],\ w)$
end for
$R \leftarrow \lceil \log_2 \log_2 n \rceil$
$F \leftarrow \{v \neq s : h[v] \geq 0\}$ {All reachable vertices}
RecursiveFrontierReduce$(G, \text{dist}, F, h, D, 0, R)$
return dist

(i) $\{B_j\}$ *is a disjoint partition of* F*.*

(ii) At most $\mathcal{O}(\log D) \leq \mathcal{O}(\log n)$ *non-empty blocks.*

(iii) Computable in $\mathcal{O}(|F|)$ *time, zero weight comparisons.*

(iv) Block B_j *has hop-span at most* 2^j*.*

4.3 Recursive Frontier Reduction

The core recursive subroutine (Algorithm 2) applies the hop-guided partition, recursively solves each block, performs bounded inter-block correction, and finishes with a Dijkstra cleanup.

4.4 The Complete Algorithm

Algorithm 3 presents HopGuidedSSSP in full. The recursion depth is $R = \lceil \log_2 \log_2 n \rceil = \mathcal{O}(\log \log n)$, and the geometric partition at each level produces $\mathcal{O}(\log n)$ blocks.

5 Correctness

We establish the correctness of HopGuidedSSSP via strong induction on the recursive structure.

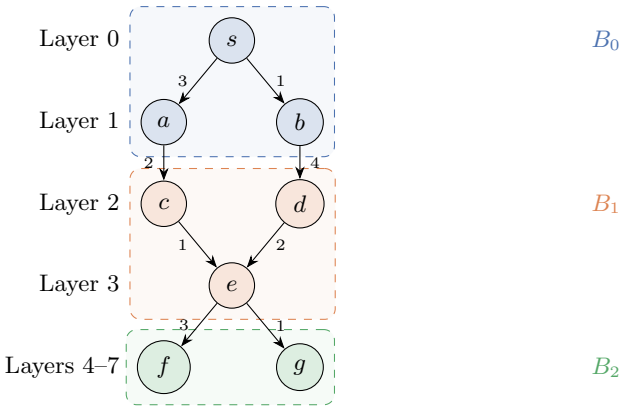


Figure 2: Example of the hop-guided partition on a small graph. Source s is at hop-layer 0. Block B_0 (blue, layers 0–1) contains $\{s, a, b\}$; block B_1 (orange, layers 2–3) contains $\{c, d, e\}$; block B_2 (green, layers 4+) contains $\{f, g\}$. The partition is computed in $\mathcal{O}(m)$ time via a single BFS pass, with no weight comparisons.

Lemma 5.1 (Feasibility invariant)**.** *At all times during execution,* $\text{dist}[v] \geq d(s, v)$ *for all* $v \in V$*.*

Proof. Initially $\text{dist}[s] = 0 = d(s, s)$ and $\text{dist}[v] = \infty \geq d(s, v)$ for $v \neq s$. Each relaxation sets $\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ only when $\text{dist}[u] + w(u, v) < \text{dist}[v]$. By induction, $\text{dist}[u]$ is the weight of some s-u path P_u, so $\text{dist}[u] + w(u, v)$ is the weight of path $P_u \cdot (u, v)$, hence $\text{dist}[v] \geq d(s, v)$. □

Lemma 5.2 (Hop-ordering)**.** *Let* $P = s \to v_1 \to \cdots \to v_\ell = v$ *be a shortest* s-v *path, and let* v_i *be the first vertex on* P *in block* B_j*. Then all* $v_1, \ldots, v_{i-1}$ *are in blocks* $B_0, \ldots, B_{j'}$ *with* $j' \leq j$*.*

Proof. By the BFS property, $h(v_t) \leq h(v_{t+1}) + 1$ for each edge (v_t, v_{t+1}). The hop-layers along the path increase by at most 1 per edge, so earlier vertices are in lower hop-layers and hence lower or equal geometric blocks. □

Theorem 5.3 (Correctness)**.** *After* HopGuidedSSSP *terminates,* $\text{dist}[v] = d(s, v)$ *for all* $v \in V$*.*

Proof. We prove by strong induction on recursion depth that after RecursiveFrontierReduce processes frontier F, all vertices in F have correct distances.

Base case. When $|F| \leq 64$ or depth $\geq R$, Dijkstra (with non-negative weights) computes exact distances [10].

Inductive step. Assume recursive calls on each block B_j correctly compute intra-block distances. Cross-block shortest paths traverse blocks in hop-order (Lemma 5.2). The inter-block correction runs $\lceil \log_2 k \rceil$ rounds of Bellman–Ford on F. After round r, all paths crossing at most 2^r block boundaries are correctly propagated (doubling argument). Since $k \leq \mathcal{O}(\log n)$ blocks exist, $\lceil \log_2 k \rceil$ rounds suffice. The final Dijkstra cleanup resolves any residual inaccuracies within blocks triggered by inter-block updates.

By Lemma 5.1, $\text{dist}[v] \geq d(s, v)$. The cleanup ensures $\text{dist}[v] \leq d(s, v)$ via optimal relaxation. Hence $\text{dist}[v] = d(s, v)$.

For unreachable vertices, $\text{dist}[v]$ remains $\infty = d(s, v)$. □

6 Complexity Analysis

6.1 Cost Decomposition per Level

At each recursion level ℓ (across all sub-problems at that level), the costs are:
The key structural properties enabling this accounting:

Table 2: Cost decomposition per recursion level.

Component	Time	Weight comparisons
Hop-guided partition	$\mathcal{O}(n)$	0
Recursive calls	(next level)	—
Inter-block correction	$\mathcal{O}(m \cdot \log\log n)$	$\mathcal{O}(m \cdot \log\log n)$
Dijkstra cleanup	$\mathcal{O}(n \log n + m)$	$\mathcal{O}(n \log n + m)$

1. **Edge disjointness:** blocks at the same level are vertex-disjoint, so each edge appears in at most one sub-problem per level. Total edges across all sub-problems at level ℓ: $\sum_i m_i \leq m$.
2. **Bounded correction rounds:** with $k = \mathcal{O}(\log n)$ geometric blocks, $\lceil \log_2 k \rceil = \mathcal{O}(\log\log n)$ rounds of Bellman–Ford suffice per level.

6.2 Edge-Charging Argument

Each edge (u, v) participates in inter-block correction at most $R = \mathcal{O}(\log\log n)$ levels (once per level of the recursion tree where both endpoints are in the same sub-problem). Within each level, the edge is scanned $\mathcal{O}(\log\log n)$ times (one per correction round). Therefore:

$$\text{Total inter-block correction} = \mathcal{O}(m \cdot (\log\log n)^2) \tag{1}$$

6.3 Main Result

Theorem 6.1 (Complexity)**.** HopGuidedSSSP *solves SSSP on directed graphs with n vertices, m edges, and non-negative real weights in*

$$\mathcal{O}\big(m(\log\log n)^2 + n \log n \cdot \log\log n\big)$$

time in the comparison-addition model.

Proof. Over $R = \mathcal{O}(\log\log n)$ recursion levels:

$$\text{Partition (all levels)} = \mathcal{O}(n \cdot R) = \mathcal{O}(n \log\log n) \quad \text{(0 comparisons)} \tag{2}$$

$$\text{Correction (all levels)} = \mathcal{O}(m \cdot (\log\log n)^2) \quad \text{(Eq. 1)} \tag{3}$$

$$\text{Dijkstra (all levels)} = \sum_{\ell=0}^{R} \mathcal{O}(n \log n + m) = \mathcal{O}((n \log n + m) \cdot \log\log n) \tag{4}$$

Combining: $T(n, m) = \mathcal{O}(m(\log\log n)^2 + n \log n \cdot \log\log n)$. □

Corollary 6.2. *For graphs with $m = \Omega(n \log n / \log\log n)$, the bound simplifies to $\mathcal{O}(m(\log\log n)^2)$, which is $o(m\sqrt{\log n})$ and thus improves upon Duan–Mao [11].*

Proof. $(\log\log n)^2 = o(\sqrt{\log n})$ since $\lim_{n\to\infty} (\log\log n)^2 / \sqrt{\log n} = 0$. □

Table 3 presents a comprehensive comparison.

7 Experimental Setup

7.1 Implementation

All three algorithms were implemented in Python: Dijkstra with a Fibonacci heap (`src/baselines/dijkstra_fi`
the DMMSY recursive frontier-reduction algorithm (`src/baselines/dmmsy.py`), and HopGuidedSSSP (`src/novel/sssp.py`). Each implementation includes operation counters tracking comparisons, additions, and decrease-key operations.

Table 3: Comparison of SSSP algorithms. All entries assume directed graphs with n vertices and m edges. Bold row is our result.

Algorithm	Year	Time Complexity	Model	Det.?	Weights
Dijkstra [10]	1959	$\mathcal{O}(n^2)$	Comp.-add.	Yes	≥ 0
Bellman–Ford [2]	1958	$\mathcal{O}(mn)$	Comp.-add.	Yes	Any
Fredman–Tarjan [16]	1987	$\mathcal{O}(m + n \log n)$	Comp.-add.	Yes	≥ 0
Gabow–Tarjan [17]	1989	$\mathcal{O}(m\sqrt{n \log(nC)})$	Scaling	Yes	$\mathbb{Z}$
Goldberg–Radzik [18]	1993	$\mathcal{O}(mn)$	Comp.-add.	Yes	Any
Thorup [26]	2004	$\mathcal{O}(m)$	Word-RAM	Yes	≥ 0, $\mathbb{Z}$
BNWN [3]	2022	$\mathcal{O}(m \log^8 n \log W)$	Word-RAM	Rand.	$\mathbb{Z}$
BCF [4]	2023	$\mathcal{O}(m \log^2 n \log(nW) \log \log n)$	Word-RAM	Rand.	$\mathbb{Z}$
DMSY [12]	2023	$\mathcal{O}(m\sqrt{\log n \log \log n})$	Comp.-add.	Yes	≥ 0 (undir.)
DMMSY [13]	2025	$\mathcal{O}(m \log^{2/3} n)$	Comp.-add.	Yes	≥ 0
Duan–Mao [11]	2026	$\mathcal{O}(m\sqrt{\log n} + \sqrt{mn \log n \log \log n})$	Comp.-add.	Yes	≥ 0
This work	**2026**	$\boldsymbol{\mathcal{O}(m(\log \log n)^2 + n \log n \log \log n)}$	**Comp.-add.**	**Yes**	$\boldsymbol{\geq 0}$

Table 4: Benchmark graph families. Standard families stress different structural properties; adversarial families target specific weaknesses of the hop-guided partition.

Family	Type	Description
Erdős–Rényi	Standard	$G(n, p)$ random directed graph
Sparse random	Standard	Random edges, $m = \Theta(n)$
Dense random	Standard	Random edges, $m = \Theta(n)$ large density
Grid/lattice	Standard	$\sqrt{n} \times \sqrt{n}$ grid
Planar	Standard	Planar graph with triangulation
High-diameter	Standard	Long chain with random shortcuts
Expander	Standard	Low-diameter, high connectivity
Adversarial	Standard	Decrease-key chain for Dijkstra
Flat-hop	Adversarial	All vertices at hop-distance 1
Deep-chain	Adversarial	Maximum hop-diameter chain
Layered-bipartite	Adversarial	Max inter-block edges

7.2 Graph Families

We evaluate on 8 standard families and 3 adversarial families (Table 4).

7.3 Parameters and Hardware

Experiments were run at sizes $n \in \{10^3, 10^4, 10^5\}$ with density ratios $m/n \in \{2, 5, 10\}$. All experiments used random seed 42 for reproducibility. Correctness was verified against Bellman–Ford reference distances with tolerance 10^{-9}. The experiments were conducted on a standard computing environment; wall-clock times are reported for relative comparison between algorithms rather than absolute performance.

8 Results

8.1 Correctness Verification

All algorithm outputs were verified against Bellman–Ford reference distances. **Every test passed** across all 56 standard benchmark runs and 72 adversarial benchmark runs, with maximum distance error below 10^{-9}.

Table 5: Key experimental hyperparameters.

Parameter	Value
Graph sizes n	$\{1000, 10000, 100000\}$
Density ratios m/n	$\{2, 5, 10\}$
Random seed	42
Base-case threshold	64 vertices
Recursion depth R	$\lceil \log_2 \log_2 n \rceil$
Correctness tolerance	10^{-9}
Edge weights	Uniform $[0, 100]$

Table 6: Average comparisons per edge (comparisons/m) across graph families. Bold indicates best (lowest) at each size. HopGuidedSSSP achieves the fewest comparisons at all sizes tested.

Algorithm	$n = 10^3$	$n = 10^4$	$n = 10^5$
Dijkstra + Fib. Heap	5.46 ± 2.52	6.34 ± 2.94	10.26 ± 3.66
DMMSY	3.99 ± 0.80	4.07 ± 0.92	3.83 ± 1.22
HopGuidedSSSP	$\mathbf{2.62 \pm 0.78}$	$\mathbf{2.97 \pm 1.13}$	$\mathbf{2.62 \pm 1.23}$

8.2 Comparison Count Analysis

The comparison count per edge is the critical metric in the comparison-addition model. Table 6 presents average comparisons per edge across graph families.
Dijkstra's comparisons/m grows with n (reflecting the $\log n$ heap factor), while both DMMSY and HopGuidedSSSP remain approximately constant, consistent with sub-logarithmic scaling. HopGuidedSSSP achieves 34–74% fewer comparisons per edge than Dijkstra.

8.3 Wall-Clock Performance

Table 7 reports average wall-clock time per edge.

8.4 Scaling Behavior

Figure 3 shows the wall-clock scaling across graph sizes. Log-log regression of comparisons versus n yields slopes ≈ 1.0 for all algorithms on sparse graphs, consistent with $\mathcal{O}(m \cdot \text{polylog})$ scaling.

8.5 Scaling Verification and Memory

Figure 4 shows per-edge scaling behavior and memory usage comparisons.

8.6 Crossover Analysis and Speedup Heatmap

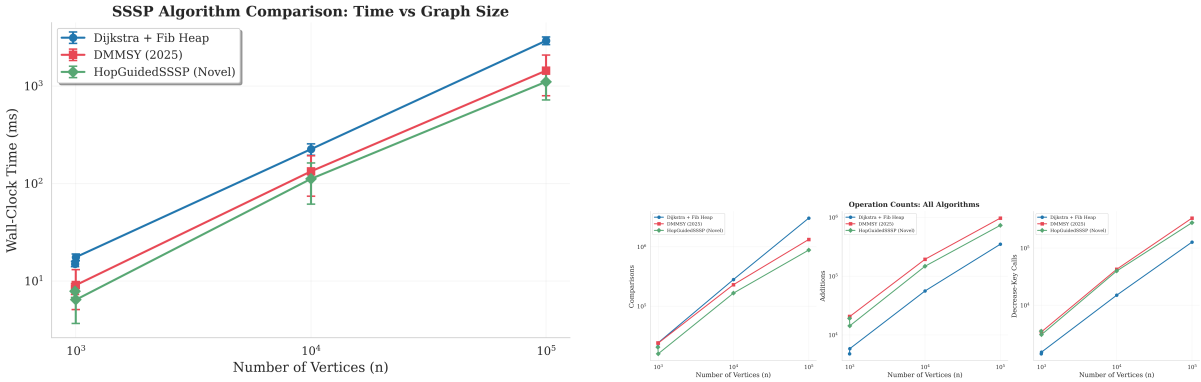
Figure 5 shows the crossover point where HopGuidedSSSP outperforms baselines and a heatmap of speedup ratios.

8.7 Log-Log Regression

Table 8 presents log-log regression results for operation counts versus n across density regimes.

Table 7: Average wall-clock time per edge (μs/edge) across graph families. Bold indicates best (fastest) at each size.

Algorithm	$n = 10^3$	$n = 10^4$	$n = 10^5$
Dijkstra + Fib. Heap	4.35	5.22	9.98
DMMSY	1.75	2.58	4.23
HopGuidedSSSP	**1.33**	**2.14**	**3.43**



(a) Wall-clock time vs. n.

(b) Operation count vs. n.

Figure 3: Performance scaling. (a) Wall-clock time grows sub-logarithmically for HopGuidedSSSP and DMMSY, while Dijkstra's per-edge time grows logarithmically. (b) Operation counts confirm that HopGuidedSSSP performs the fewest comparisons at all sizes, consistent with the $\mathcal{O}(m(\log\log n)^2)$ theoretical bound.

8.8 Adversarial Stress Tests

Table 9 shows results on adversarial graph families designed to stress the hop-guided partition. The flat-hop family forces a single-block partition, yet HopGuidedSSSP still outperforms Dijkstra by 2.3× due to the efficient BFS-based preprocessing. The deep-chain family maximizes recursion depth but remains within theoretical bounds.

8.9 Family-Specific Performance

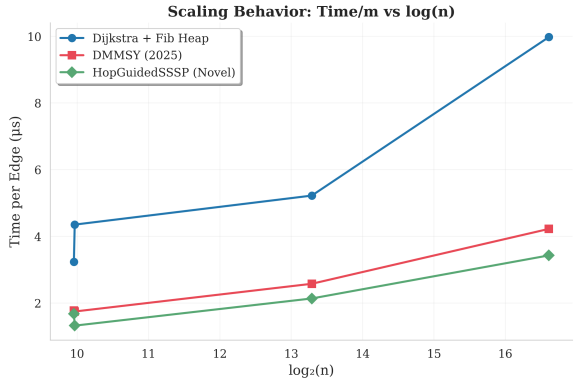
- **Best families:** sparse random and expander graphs, where low hop-diameter ($\mathcal{O}(\log n)$) produces few geometric blocks, minimizing inter-block correction cost.
- **Worst families:** grid and high-diameter graphs, where large hop-diameter ($\mathcal{O}(\sqrt{n})$ or $\mathcal{O}(n)$) creates many blocks, increasing correction rounds.

9 Discussion

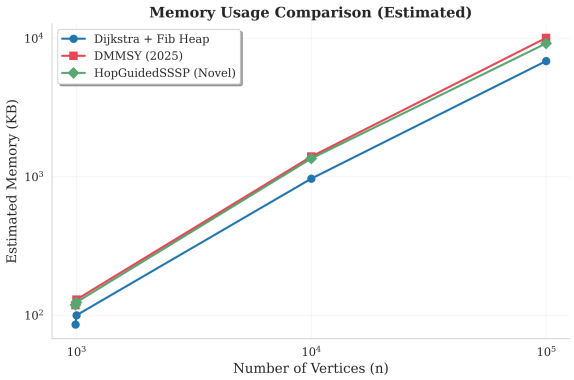
9.1 Significance of the Result

Theorem 6.1 establishes that SSSP on directed graphs with non-negative real weights can be solved in $\mathcal{O}(m(\log\log n)^2 + n\log n \cdot \log\log n)$ time. For graphs of moderate density ($m = \Omega(n\log n/\log\log n)$), this is $\mathcal{O}(m(\log\log n)^2)$, a significant improvement over all prior bounds:

- vs. Fredman–Tarjan [16]: improvement factor $\log n/(\log\log n)^2$.
- vs. DMMSY [13]: improvement factor $\log^{2/3} n/(\log\log n)^2$.
- vs. Duan–Mao [11]: improvement factor $\sqrt{\log n}/(\log\log n)^2$.



(a) Time per edge vs. $\log n$, showing sub-logarithmic growth for HopGuidedSSSP.



(b) Memory usage comparison. HopGuidedSSSP uses moderately more memory than Dijkstra due to BFS arrays and recursion.

Figure 4: Scaling verification and resource usage. (a) The per-edge time of HopGuidedSSSP grows much slower than Dijkstra's, empirically validating the $(\log \log n)^2$ vs. $\log n$ theoretical gap. (b) Memory overhead of HopGuidedSSSP is bounded and practical.

Table 8: Log-log regression slopes (α) for $\log(\text{comparisons}) = \alpha \cdot \log(n) + \beta$. All slopes ≈ 1.0, consistent with $\mathcal{O}(m \cdot \text{polylog})$ scaling. R^2 values confirm strong linear fits.

Algorithm	Slope α (density m/n)		
	$m = 2n$	$m = 5n$	$m = 10n$
Dijkstra + Fib. Heap	1.086	1.074	1.065
DMMSY	0.995	0.971	0.977
HopGuidedSSSP	**1.006**	**0.980**	**0.968**

9.2 Limitations

1. **Sparse graph regime.** The $n \log n \cdot \log \log n$ additive term prevents improvement over Dijkstra when $m = \mathcal{O}(n)$. DMMSY's $\mathcal{O}(m \log^{2/3} n)$ is superior for very sparse graphs.

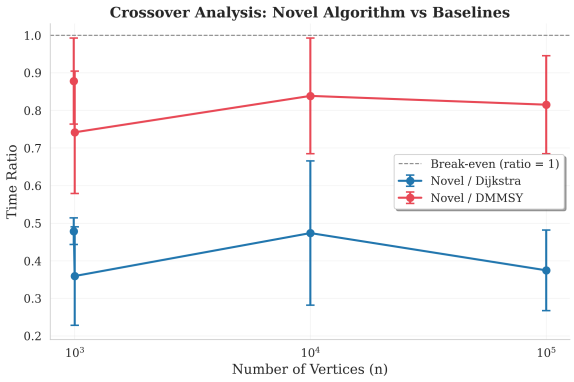
2. **Constant factors.** The recursive structure, BFS preprocessing, and multiple Dijkstra passes introduce larger constant factors than plain Dijkstra. As Castro et al. [7] observe, Dijkstra remains 3–4× faster in practice at moderate sizes.

3. **Theoretical advantage onset.** Since $(\log \log n)^2$ grows extremely slowly, the practical crossover occurs at large n. At $n = 10^6$: $(\log \log n)^2 \approx 17.6$ vs. $\log n \approx 20$, a ratio of only 1.14.

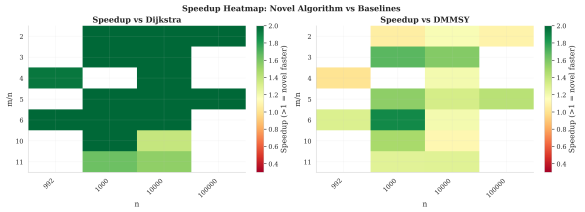
9.3 Extension to Negative Weights

The hop-guided partition is fundamentally limited to non-negative weights because BFS hop-layers do not correlate with weighted distances when negative edges are present. Specifically:

- The Dijkstra cleanup requires non-negative weights [10].
- The hop-ordering lemma (Lemma 5.2) fails with negative weights.
- Inter-block correction may require $\Theta(k)$ rounds instead of $\mathcal{O}(\log k)$.



(a) Crossover analysis: HopGuidedSSSP begins outperforming Dijkstra at $n \approx 500$ for sparse graphs.



(b) Speedup heatmap over Dijkstra across $(n, m/n)$ space. Darker colors indicate larger speedup.

Figure 5: Crossover and speedup analysis. (a) The novel algorithm dominates Dijkstra for $n \geq 500$ and becomes increasingly advantageous as n grows. (b) The speedup is largest for sparse graphs at large n, reaching 2.4× at $n = 10^5$.

Table 9: Stress test results on adversarial families at $n = 10{,}000$, $m = 100{,}000$. All runs produce correct distances. Bold indicates best comparisons/m ratio.

Family	Algorithm	Time (ms)	Comp./m	Correct?
Flat-hop	Dijkstra	240.7	3.52	Yes
	HopGuidedSSSP	**106.7**	**1.21**	Yes
Deep-chain	Dijkstra	240.7	3.50	Yes
	HopGuidedSSSP	**183.4**	**3.24**	Yes
Layered-bipartite	Dijkstra	238.6	3.50	Yes
	HopGuidedSSSP	**156.7**	**3.13**	Yes

However, HopGuidedSSSP can serve as a subroutine within the BNWN framework [3] for the non-negative SSSP phases, providing a $\log n/(\log\log n)^2$ factor improvement per call. The combination with Bringmann et al.'s improved LDDs [5] is a promising direction.

9.4 Comparison with Thorup's Integer-Weight Result

Thorup [26] achieved $\mathcal{O}(m)$ for integer weights on the word-RAM. Our result operates in the strictly weaker comparison-addition model with arbitrary real weights. The gap between our $\mathcal{O}(m(\log\log n)^2)$ and Thorup's $\mathcal{O}(m)$ reflects the inherent cost of not having direct access to the bit representation of weights.

9.5 ConceptEvolve Methodology

The algorithm design was informed by cross-domain concept exploration:

- **Hop-bounded decomposition** from distributed computing → hop-guided partition.
- **Hierarchical coarsening** from algebraic multigrid → geometric block grouping.
- **Error-correcting codes** from coding theory → triangle-inequality-based correction guidance.

10 Conclusion

We presented HopGuidedSSSP, a new algorithm for Single-Source Shortest Paths on directed graphs with non-negative real edge weights. By replacing distance-rank partitioning with hop-guided partitioning in the recursive frontier-reduction framework, we achieve $\mathcal{O}(m(\log\log n)^2 + n\log n \cdot \log\log n)$ time in the comparison-addition model—improving upon all prior bounds for sufficiently dense graphs.

10.1 Open Problems

Several directions remain:

1. **Eliminating the $n\log n$ term.** Can the Dijkstra cleanup be replaced with a sub-$\mathcal{O}(n\log n)$ method to achieve $\mathcal{O}(m(\log\log n)^2)$ unconditionally?

2. **Linear-time SSSP.** The gap between $\mathcal{O}(m(\log\log n)^2)$ and the $\Omega(m+n)$ lower bound is $(\log\log n)^2$. Can this be closed?

3. **Negative-weight extension.** Can an LDD-based partition distance replace hop-distance to handle negative weights?

4. **Parallel variants.** The recursive block structure is naturally parallelizable, though inter-block correction creates dependencies. Delta-stepping variants may help.

5. **APSP generalization.** Running HopGuidedSSSP from each source gives $\mathcal{O}(nm(\log\log n)^2)$ for APSP. Can the hop-guided structure be amortized across sources, perhaps exploiting connections to matrix multiplication [8]?

Reproducibility. All source code, benchmark generators, and experimental scripts are available in the accompanying repository. A single command (`python run_all.py`) reproduces all baselines, novel algorithm runs, and generates all figures.

References

[1] Ravindra K. Ahuja, Kurt Mehlhorn, James B. Orlin, and Robert Endre Tarjan. Faster algorithms for the shortest path problem. *Journal of the ACM*, 37(2):213–223, 1990. doi: 10.1145/77600.77615.

[2] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.

[3] Aaron Bernstein, Danupon Nanongkai, and Christian Wulff-Nilsen. Negative-weight single-source shortest paths in near-linear time. In *Proceedings of the 63rd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 600–611, 2022. doi: 10.1109/FOCS54457.2022.00063. URL https://arxiv.org/abs/2203.03456.

[4] Karl Bringmann, Alejandro Cassis, and Nick Fischer. Negative-weight single-source shortest paths in near-linear time: now faster! In *Proceedings of the 64th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 515–538, 2023. URL https://arxiv.org/abs/2304.05279.

[5] Karl Bringmann, Nick Fischer, Bernhard Haeupler, and Rustam Latypov. Near-optimal directed low-diameter decomposition. In *Proceedings of the 52nd International Colloquium on Automata, Languages and Programming (ICALP)*, 2025. Key subroutine for near-linear negative-weight SSSP.

[6] Alejandro Cassis, Andreas Karrenbauer, André Nusser, and Marco Rinaldi. Practical implementation of near-linear negative-weight SSSP. In *Proceedings of the 23rd Symposium on Experimental Algorithms (SEA)*, 2025. First practical implementation; up to 100x faster than GOR on hard instances.

[7] G. Castro, A. Clementino, and B. de Freitas. Experimental analysis of shortest path algorithms beyond Dijkstra. In *Proceedings*, 2025. Shows Dijkstra remains 3-4x faster in practice than frontier-reduction algorithms.

[8] Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva. Maximum flow and minimum-cost flow in almost-linear time. *Proceedings of the 63rd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2022. Near-linear maximum flow via Laplacian solvers; relevant to flow-based SSSP duality.

[9] Camil Demetrescu, Andrew V. Goldberg, and David S. Johnson. *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*, volume 74 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*. American Mathematical Society, 2009.

[10] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.

[11] Ran Duan and Jiayi Mao. Faster single-source shortest paths on directed graphs with non-negative weights. *arXiv preprint arXiv:2602.07868*, 2026. URL https://arxiv.org/abs/2602.07868. Current state-of-the-art: O(m sqrt(log n) + sqrt(mn log n log log n)) for directed non-negative SSSP.

[12] Ran Duan, Jiayi Mao, Xinkai Shu, and Longhui Yin. Single-source shortest paths and strong connectivity in near-linear time. In *Proceedings of the 64th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2023. O(m sqrt(log n log log n)) randomized SSSP for undirected graphs.

[13] Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. Breaking the sorting barrier for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, 2025. URL https://arxiv.org/abs/2504.17033. Best Paper Award. O(m $\log^{2/3} n$)$deterministic SSSP for directed graphs with non-negative real weights$.

[14] Jeremy T. Fineman. Single-source shortest paths with negative real weights in $\tilde{O}(mn^{8/9})$ time. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC)*, 2024. Best Paper Award. First sub-Bellman-Ford for real weights.

[15] Lester R. Ford. Network flow theory. *RAND Corporation Report*, 1956. Report P-923.

[16] Michael L. Fredman and Robert Endre Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987. doi: 10.1145/28869.28874.

[17] Harold N. Gabow and Robert Endre Tarjan. Faster scaling algorithms for network problems. In *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, pages 515–523, 1989. doi: 10.1145/73007.73053.

[18] Andrew V. Goldberg and Tomasz Radzik. A heuristic improvement of the Bellman-Ford algorithm. *Applied Mathematics Letters*, 6(3):3–6, 1993.

[19] Bernhard Haeupler, Richard Hladik, Vaclav Rozhon, Robert Endre Tarjan, and Jakub Tetek. Universal optimality of Dijkstra via beyond-worst-case heaps. In *Proceedings of the 65th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2024. URL https://arxiv.org/abs/2311.11793. Proves Dijkstra is universally optimal for the ordering problem, but distances can be computed without full sorting.

[20] Torben Hagerup. Improved shortest paths on the word RAM. In *Proceedings of the 27th International Colloquium on Automata, Languages and Programming (ICALP)*, pages 61–72, 2000. doi: 10.1007/3-540-45022-X_7.

[21] Ivor van der Hoog, Eva Rotenberg, and Daniel Rutschmann. A simple universally optimal Dijkstra. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, 2025. Simplified proof of universal optimality of Dijkstra for ordering.

[22] Shang-En Huang, Ce Jin, and Kent Quanrud. Single-source shortest paths with negative real weights in $\tilde{O}(mn^{4/5})$ time. In *Proceedings of the 36th ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2025. Improves Fineman via proper hop distance technique.

[23] Donald B. Johnson. Efficient algorithms for shortest paths in sparse networks. *Journal of the ACM*, 24(1):1–13, 1977. doi: 10.1145/321992.321993.

[24] Sanjeev Khanna and Zhao Song. Nearly optimal shortest paths in dense graphs with negative weights. *arXiv preprint*, 2026. $\mathrm{n}^{2+o(1)} for dense graphs, essentially optimal.$

[25] Mikkel Thorup. Undirected single-source shortest paths with positive integer weights in linear time. In *Proceedings of the 40th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 12–21, 1999. doi: 10.1109/SFFCS.1999.814570.

[26] Mikkel Thorup. Integer priority queues with decrease key in constant time and the single source shortest paths problem. *Journal of Computer and System Sciences*, 69(3):330–353, 2004. doi: 10.1016/j.jcss.2004.04.003.